Department of Computer Engineering

St. Francis Institute of Technology

University of Mumbai

2026-2027



A Mini-Project Report On

"Detecting Coordinated and Suspicious Reviewing Behaviour in 41 Million Amazon Reviews using DuckDB, Parquet and Next.js"

Subject - Big Data Analysis

Under the guidance of

Ms. Monalisa Lopes

By

Aditya Nair (25, 232110)

Arya Patil (27, 232117)

Siddhi Trivedi(38, 232170)

TABLE OF CONTENTS

Abstract i

List of abbreviations ii

List of tables iii

List of figures iii

Chapter 1: Introduction 1

1.1 Introduction 1
1.2 Problem definition 1
1.3 Objectives 1

Chapter 2: Proposed work / methodology 2

2.1 System architecture 2
2.2 Data preprocessing 3
2.3 Models 3
2.4 Algorithms 4

Chapter 3: Experimental setup 5

3.1 Dataset description 5
3.2 Storage layout (Parquet and DuckDB) 6
3.3 Tools 6
3.4 Libraries and frameworks 7

Chapter 4: Implementation 7

4.1 Backend implementation 7
4.2 Important functions 7
4.3 Next.js dashboard 10

Chapter 5: Results 11

5.1 Results 11
5.2 Discussion 18
5.3 Limitations 18

Chapter 6: Conclusion 19

Chapter 7: References 19

Abstract

Online marketplaces depend on customer reviews, and coordinated groups of accounts can distort them. This project analyses the complete 2014 Amazon review corpus (McAuley/UCSD, 5-core) — 41,135,700 reviews written by 3,035,045 accounts about 1,569,973 products between 1996 and 2014, a 9.87 GiB compressed file that expands to about 44 GB of JSON — to find accounts whose behaviour suggests they are not acting independently. Because no review in the corpus is labelled, the system is unsupervised. The raw file is converted once into columnar Parquet storage and processed out-of-core with the DuckDB analytical engine on a single laptop with 15.4 GB of RAM. Three independent families of evidence are computed for every account: copied text, found with MinHash and locality-sensitive hashing and confirmed by exact Jaccard similarity; unusual activity, measured against a de-trended daily baseline; and co-reviewing in packs, found in a sparse co-review graph of 1,038,862 accounts. Twenty-one features are converted to empirical percentiles and rank-aggregated into a consensus score and an any-family score. The pipeline identified 1,381 dense groups of accounts reviewing the same products within days of each other, 79,938 verified near-duplicate text clusters, and 179 accounts flagged by more than one family at once. Validation showed that text features alone predict behavioural and graph suspicion with 72.1% accuracy (ROC AUC 0.805), and a planted-campaign study exposed and fixed a blind spot in the graph detector. An interactive Next.js dashboard built with shadcn/ui presents the findings, including a network view of the most suspicious accounts.

Keywords: big data, DuckDB, Parquet, review spam, MinHash, locality-sensitive hashing, graph analytics, unsupervised anomaly detection, Next.js

List of abbreviations

API – Application Programming Interface
AUC – Area Under the Curve
CSR – Compressed Sparse Row (sparse matrix format)
ECDF – Empirical Cumulative Distribution Function
GiB – Gibibyte (2^{30} bytes)
JSON – JavaScript Object Notation
LOO – Leave-One-Out
LSH – Locality-Sensitive Hashing
NDJSON – Newline-Delimited JSON
RAM – Random Access Memory
ROC – Receiver Operating Characteristic
RSS – Resident Set Size (memory in use)
SQL – Structured Query Language
UI – User Interface
ZSTD – Zstandard compression

List of tables

Table 3.1: Dataset statistics 5
Table 3.2: Features used in the suspicion score 6
Table 3.3: Libraries and frameworks used 7
Table 5.1: Run time and peak memory of each pipeline stage 11
Table 5.2: Exact and near-duplicate text detection 15
Table 5.3: Disclosure of free or discounted products, by group 17
Table 5.4: Validation results 18

List of figures

Figure 2.1: System architecture of the project 2
Figure 4.1: Single-pass ingest query 8
Figure 4.2: MinHash permutations and signature() 8
Figure 4.3: Ring detection: connected components filtered by size and density 9
Figure 4.4: ECDF transform and rank aggregation 10
Figure 5.1: Dashboard overview (Start here page) 11
Figure 5.2: Distribution of the suspicion score, with an adjustable cut-off 12
Figure 5.3: Rating behaviour by suspicion decile 13
Figure 5.4: Overlap between the three families' top 10,000 accounts 14
Figure 5.5: All 1,381 rings by lifespan and density 14
Figure 5.6: Convergence plot of ring #179919 15
Figure 5.7: Near-duplicate cluster #60821, shown side by side 16
Figure 5.8: Network view: the small, separate groups 16
Figure 5.9: Network view: the linked core of 2,479 accounts 17

Chapter 1: Introduction

1.1 Introduction

Customer reviews are one of the main signals people use when they buy online. A product with hundreds of five-star reviews looks trustworthy, which makes reviews worth faking. Fake or coordinated reviews are rarely the work of one account: they are usually posted by groups that review the same products at the same time, reuse the same wording, or behave in ways ordinary shoppers do not.

Finding such groups is a big-data problem. The dataset used here holds 41,135,700 reviews from 3,035,045 accounts, and the interesting patterns only appear when every account is compared with every other: which accounts share text, which ones keep reviewing the same products within days of each other, and whose activity is out of line with the rest. That means near-duplicate search over 41 million documents and a graph of more than 11 million account pairs, which is far beyond what a spreadsheet or an in-memory script can handle.

This project builds a complete pipeline that does this on a single laptop, with no cloud services. The source file is converted once into compressed columnar Parquet files and analysed with DuckDB, an embedded analytical database that processes data larger than memory by spilling to disk [11]. The results are served by a small FastAPI service to an interactive dashboard built with Next.js and shadcn/ui [16].

1.2 Problem definition

The corpus contains no labels: nobody has marked which reviews are genuine. It is also missing features that published fake-review detectors rely on [3]–[7], [14]. The 5-core filtering removed every account with fewer than five reviews, the 2014 release has no verified-purchase flag, and no product metadata is available. The project therefore cannot claim to find “fake reviews”. The question it can answer is narrower and defensible:

“To design and implement a scalable, unsupervised system that identifies accounts whose reviewing behaviour suggests they are not acting independently — through copied text, unusual activity, or coordinated co-reviewing — across all 41.1 million reviews, and to present the evidence in an interactive dashboard.”

1.3 Objectives

- To convert the 9.87 GiB compressed source into columnar Parquet storage in a single pass, and to process it out-of-core with DuckDB on a 15.4 GB laptop.
- To profile the data quality of the corpus and build a de-trended daily baseline, since review volume grew more than a thousand-fold between 1997 and 2013.
- To detect exact and near-duplicate review text at scale using MinHash and locality-sensitive hashing, verifying every candidate with exact Jaccard similarity.
- To build a co-review graph of accounts and detect small, dense groups (rings) that review the same products within days of each other.

- To compute behavioural features such as review bursts, account lifespan and leave-one-out rating deviation for every account.
- To combine three independent families of evidence into a suspicion score with an explanation for every account, without using any rating level or helpful vote.
- To validate the score without ground truth, using a cross-family check, a helpful-vote proxy and planted synthetic campaigns.
- To build an interactive Next.js dashboard that explains the findings to a first-time reader, with charts, a group explorer and a network view of the most suspicious accounts.

Chapter 2: Proposed work / methodology

2.1 System architecture

The system is a staged pipeline (Figure 2.1). The compressed source is read exactly once and written to Parquet, together with hashes and text counters computed in the same pass. A narrow fact table with integer surrogate keys is then derived, and three independent families of evidence are computed from it: text, behaviour and graph. Stage 9 merges them into scores for all 3,035,045 accounts. Every stage writes its results to Parquet and checks them against measured corpus totals, stopping the build on any mismatch, so a partial or corrupted result cannot reach the dashboard. A FastAPI service exposes the precomputed results to a Next.js dashboard; both listen only on 127.0.0.1.

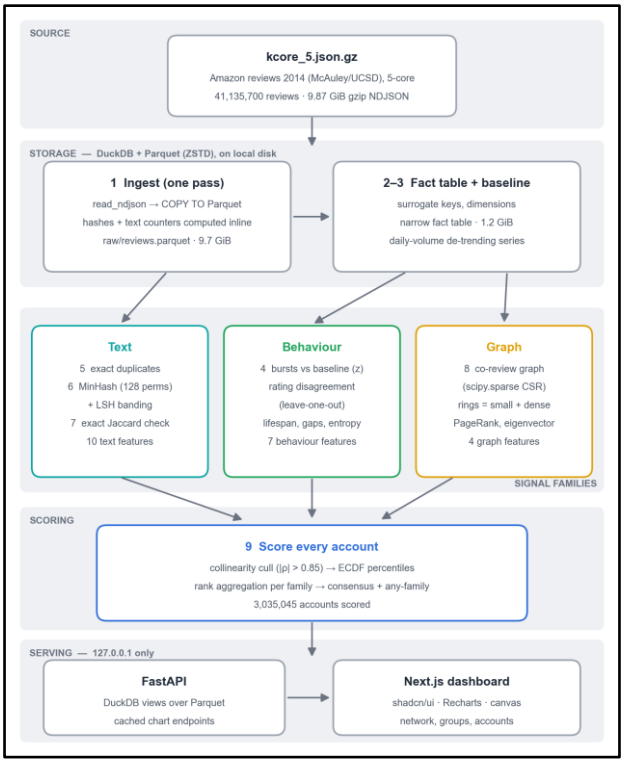


Figure 2.1 — System architecture of the project

2.2 Data preprocessing

The source is a gzip-compressed newline-delimited JSON file. Because gzip decompresses serially at about 210 MB/s regardless of thread count, reading it is the hard floor on ingest time, so every per-review value that needs the raw text is computed during that single pass. The steps are:

- **Single-pass ingest** – DuckDB's `read_ndjson` streams the file straight into a ZSTD-compressed Parquet file of 9.7 GiB; the source is never read again.
- **Duplicate keys** – a hash of the raw text, and a hash of the normalised text (lowercased, whitespace collapsed) that catches trivially reformatted copies.
- **Text counters** – characters, words, exclamation marks and capital letters, and whether the review title repeats the body.
- **Surrogate keys and fact table** – reviewer and product identifiers are replaced by integers, giving a narrow 1.2 GiB fact table that later stages read instead of the text.
- **Validation** – row, reviewer, product and rating counts are asserted against measured totals; 4 missing timestamps and 8,704 empty texts are excluded rather than imputed.
- **De-trending baseline** – daily review volume over 6,266 days, so that a burst is judged against what was normal for that date rather than against the 2013 peak.

Documents shorter than 20 words (808,313, or 1.96%) are excluded from near-duplicate detection, because short texts collide by chance rather than by copying.

2.3 Models

2.3.1 Columnar storage and out-of-core processing

Parquet stores each column separately and compresses it, so a query that needs three columns reads only those three [15]. DuckDB executes SQL over Parquet files in parallel and spills to disk when an operation does not fit in memory [11]. Together they let a single machine process the full corpus: every stage is SQL or NumPy over Parquet, intermediate results are written directly to Parquet rather than held in memory, and the largest stage peaked at 8.1 GB of RAM.

2.3.2 Three independent families of evidence

Each account is described by features in three families that share no input:

- **Copied writing (text, 10 features)** – how often the account's text duplicates or nearly duplicates other reviews, the size of the largest such cluster, and style measures such as length variation and exclamation rate.
- **Unusual activity (behaviour, 7 features)** – the busiest 7-day window, reviews per active day, lifespan, regularity of gaps, rating entropy, leave-one-out rating deviation and how early the account reviews products.
- **Moves in a pack (graph, 4 features)** – co-review weight, the size and density of the account's ring, and the centrality of the products it reviews.

Keeping the families disjoint is deliberate: if they later agree about an account, that agreement is evidence, not an artefact of shared inputs.

2.3.3 Suspicion scoring

Features are on very different scales and many are heavy-tailed, so each is converted to its empirical percentile (ECDF) across all accounts. Within a family the percentiles are averaged; across families they are ranked again so that no family dominates. Two scores are produced: the **consensus score**, the mean of the three family ranks, which is high only when several families agree; and the **any-family score**, the maximum, which casts a wider net with weaker evidence per account. Rating levels and helpful votes are excluded by contract: 59.21% of all reviews are five-star, so a rating level would flag three fifths of everyone, and helpful votes are kept back for validation.

2.4 Algorithms

2.4.1 MinHash and locality-sensitive hashing

Comparing every pair of 41 million reviews is impossible (about 8.5×10^{14} pairs). Each review is instead turned into a set of 5-word shingles, and 128 MinHash values are computed with seeded multiply-shift hash functions [8]. The probability that two reviews share a MinHash value equals their Jaccard similarity. The 128 values are cut into bands, and reviews that collide in any band become candidates [9]. Two banding configurations are used: loose (16 bands of 8, threshold $J \approx 0.707$) and tight (8 bands of 16, $J \approx 0.878$). Every candidate is then verified by computing the exact Jaccard similarity of its shingle set against its cluster exemplar; 17.7% of loose candidates failed this check.

2.4.2 Co-review graph and ring detection

Two accounts are linked when they reviewed at least three of the same products within 30 days of each other. The resulting graph of 1,038,862 accounts and 11,181,701 links is held as a SciPy sparse CSR matrix. Connected components alone do not work on this graph: at every threshold one giant component swallows 82–93% of the accounts, because a few prolific reviewers bridge everyone. A ring is therefore defined as a component that is both small (3–200 accounts) and dense (at least 30% of all possible links present). Product centrality is computed with PageRank [10] and eigenvector centrality by power iteration.

2.4.3 ECDF rank aggregation and collinearity cull

Before scoring, highly correlated features are culled (Spearman $|\rho| > 0.85$), measured only on accounts where at least one of the pair is non-zero; otherwise features that are zero for 99% of accounts look perfectly correlated. Three features were removed, leaving 21 of 24. The remaining features are converted to ECDF percentiles, with ties sharing a percentile, and aggregated as described in Section 2.3.3.

2.4.4 Evaluation without ground truth

With no labels, the score is checked in three independent ways:

- **Cross-family check** – a gradient-boosted classifier tries to predict, from text features alone, whether an account is in the top or bottom decile of behaviour-plus-graph suspicion. Since the families share no input, success means they detect a common phenomenon.
- **Helpful-vote proxy** – ROC AUC against accounts whose reviews are mostly voted unhelpful, the labelling rule used by the REV2 and CARE-GNN Amazon benchmarks [12], [13].
- **Planted campaigns** – 360 synthetic campaigns (4,800 accounts, about 72,000 reviews) are injected and the whole pipeline re-run, giving a real recall measurement.

Chapter 3: Experimental setup

3.1 Dataset description

The project uses the Amazon product review data published by McAuley et al. [1], [2]: the 2014 release, complete review data, 5-core subset, in which every account and every product has at least five reviews. It is a static public research dataset, not live Amazon data. Its main statistics, measured over three full passes, are given in Table 3.1.

Property	Value
Source	McAuley/UCSD Amazon product data, 2014, 5-core
File	kcore_5.json.gz – 9.87 GiB gzip NDJSON (about 44.3 GB decompressed)
Reviews	41,135,700
Accounts (reviewers)	3,035,045 (min 5, median 8, max 23,851 reviews)
Products	1,569,973 (min 5, median 10, max 13,550 reviews)
Period	20 May 1996 – 23 July 2014, day-granular timestamps
Ratings	5★ 59.21% · 4★ 20.36% · 3★ 9.32% · 2★ 5.02% · 1★ 6.09%
Review length	median 52 words, mean 100 words
Missing values	932,551 reviewer names (2.27%), 8,704 empty texts, 4 timestamps
Fields	reviewerID, asin, reviewerName, helpful, reviewText, overall, summary, unixReviewTime

Table 3.1: Dataset statistics

Review volume is extremely uneven over time: 2013 alone holds 35.96% of all reviews and 1997 only 2,707. This is why every time-based feature is de-trended against the daily baseline instead of being compared in absolute terms.

Table 3.2 lists the 21 features that survived the collinearity cull, by family. Features excluded by contract (rating level, five-star share, helpful votes, raw review count) never enter the score.

Family	Features
Copied writing (10)	self_dup_rate, summary_reuse_rate, summary_is_text_rate, exact_dup_rate, neardup_rate, neardup_max_cluster, mean_words, words_cv, exclaim_rate, upper_rate
Unusual activity (7)	max_in_7d, reviews_per_active_day, lifespan_days, gap_cv, rating_entropy, loo_rating_dev, early_tendency
Moves in a pack (4)	coreview_weight, ring_size, ring_density, max_product_eigen
Removed by the cull (3)	max_per_day (ρ 0.952), coreview_degree (ρ 0.968), mean_product_pagerank (ρ 0.888)

Table 3.2: Features used in the suspicion score

3.2 Storage layout (Parquet and DuckDB)

All data lives on the local disk under `data/`, one folder per stage, as ZSTD-compressed Parquet. Stages read each other's outputs through DuckDB views, and one configuration module (`pipeline/common.py`) fixes the DuckDB settings for every stage: a 10 GB memory limit, 10 threads, and a temporary directory on the D: drive for out-of-core spill. The raw text (9.7 GiB) is kept apart from the narrow fact table (1.2 GiB), so analyses that do not need text never read it. A single command, `python run.py`, checks which outputs exist, builds only what is missing in dependency order, and then starts the API and dashboard.

3.3 Tools

- **Hardware:** AMD Ryzen 5 5600H (6 cores, 12 threads), 15.4 GB RAM, no GPU, Windows 11.
- **Python 3.10:** the whole data pipeline and the API.
- **DuckDB 1.4:** in-process SQL engine over Parquet, with out-of-core execution.
- **Apache Parquet (ZSTD):** columnar storage for every intermediate and final result.
- **Node.js 22 and Next.js 16:** the dashboard.
- **Visual Studio Code:** source code editing.
- **Git:** version control.

3.4 Libraries and frameworks

Library / Framework	Purpose
duckdb 1.4.3	SQL over Parquet, joins, aggregation, out-of-core sorting
pyarrow 24.0	Parquet reading and streaming writes
numpy 2.2 / pandas 2.2	Vectorised MinHash, feature matrices
scipy 1.15	Sparse co-review graph, connected components, rank statistics
scikit-learn 1.6	Gradient-boosted classifier for the cross-family validation
networkx 3.4	Network layout for the dashboard's network view
xxhash 3.7	Fast 64-bit hashing of words and shingles
fastapi 0.111 / uvicorn	Local API serving precomputed results
next 16.3 / react 19.2	Dashboard framework
shadcn/ui, Tailwind CSS 4	Dashboard components and styling
recharts 3.10	Charts (area, bar, scatter, donut, heatmap)

Table 3.3: Libraries and frameworks used

Chapter 4: Implementation

4.1 Backend implementation

The backend is a set of numbered pipeline stages in `pipeline/`, each a small Python program with one job: 01 ingest, 02 fact table, 03 data quality, 04 features, 05 exact duplicates, 06 MinHash, 07 near-duplicates, 08 graph, 09 scoring, and 15 the network view. Each stage records its run time and peak memory, writes its outputs to Parquet, and asserts them against measured totals. The API in `api/` opens DuckDB views over those files and never scans the 41-million-row fact table to answer a request; every chart reads a precomputed table, and chart results are cached for the life of the process.

4.2 Important functions

4.2.1 Single-pass ingest – read_ndjson to Parquet

File: pipeline/01_ingest.py

The only stage that reads the 9.87 GiB source. Duplicate-detection hashes and text counters are computed in the same streaming query and written to ZSTD Parquet. Every stage opens DuckDB through one shared function (`pipeline/common.py`) with a 10 GB memory limit, 10 threads and a spill directory on disk. Measured: 41,135,700 rows in about 5 minutes.

pipeline/01_ingest.py

```
    -- exact-duplicate keys
    hash(reviewText)                                    AS text_hash,
    hash(summary)                                       AS summary_hash,
    -- normalised duplicate key: lowercased, whitespace-collapsed. Catches
    -- trivially-reformatted copies that a raw hash misses.
    hash(regexp_replace(lower(trim(reviewText)), '\s+', ' ', 'g')) AS norm_text_hash,

    -- style counters (the "shoutiness" factor plus length)
    length(reviewText)                                                  AS n_chars,
    length(reviewText) - length(replace(reviewText, ' ', '')) + 1       AS n_words,
    length(reviewText) - length(replace(reviewText, '!', ''))           AS n_exclaim,
    length(reviewText) - length(regexp_replace(reviewText, '[A-Z]', '', 'g')) AS n_upper,
    (lower(trim(summary)) = lower(trim(reviewText)))                    AS summary_is_text
FROM read_ndjson(
    '{common.SOURCE_GZ.as_posix()}',
    columns={COLUMNS},
    ignore_errors=true
)

con.execute(f"""
    COPY ({SELECT_SQL})
    TO '{common.RAW_PARQUET.as_posix()}'
    (FORMAT PARQUET, COMPRESSION ZSTD, COMPRESSION_LEVEL 3, ROW_GROUP_SIZE 122880)
""")
```

Figure 4.1 — Single-pass ingest query

4.2.2 signature() – MinHash

File: pipeline/06_minhash.py

Words are hashed with xxHash, combined into 5-word shingles, and 128 MinHash values are taken with seeded multiply-shift hashing, fully vectorised in NumPy. Signatures stream to disk row group by row group, which kept each worker at 66 MB of memory. Stage 7 then confirms every LSH candidate by exact Jaccard similarity before reporting it.

pipeline/06_minhash.py

```
def _permutations() -> tuple[np.ndarray, np.ndarray]:
    """Fixed multiply-shift coefficients. Seeded so the build is reproducible."""
    rng = np.random.default_rng(SEED)
    a = rng.integers(1, 1 << 62, size=N_PERMS, dtype=np.uint64) | np.uint64(1)
    b = rng.integers(0, 1 << 62, size=N_PERMS, dtype=np.uint64)
    return a, b

def signature(text: str) -> np.ndarray:
    """128 MinHash values for one document, as uint64 in [0, 2**32)."""
    words = text.lower().split()
    word_hashes = np.fromiter(map(xxhash.xxh64_intdigest, words),
                              dtype=np.uint64, count=len(words))
    if len(word_hashes) < K:
        shingles = word_hashes[:1]
    else:
        shingles = (np.lib.stride_tricks.sliding_window_view(word_hashes, K)
                    * PW_SHINGLE).sum(axis=1)
    return ((A[None, :] * shingles[:, None] + B[None, :]) >> _SHIFT).min(axis=0)
```

Figure 4.2 — MinHash permutations and signature()

4.2.3 Ring detection

File: pipeline/08_graph.py

Connected components of the co-review graph are found with SciPy, and a component is kept as a ring only if it is small enough and dense enough. Components that are too large are the graph's small-world core, reported separately rather than called a ring.

pipeline/08_graph.py

```
graph_full = build(shared_30d >= 2)     # per-reviewer degree features
graph = build(shared_30d >= EDGE_SHARED_30D)
_, labels = connected_components(graph, directed=False)
present = np.nonzero(np.diff(graph.indptr))[0]
sizes = np.bincount(labels)
degree = np.diff(graph_full.indptr)
weight = np.asarray(graph_full.sum(axis=1)).ravel() / 2.0

candidates = np.unique(labels[present])
candidates = candidates[(sizes[candidates] >= MIN_RING_SIZE)]
order = np.argsort(labels, kind="stable")
sorted_labels = labels[order]
keep, rejected_big, rejected_sparse = [], [], []
for comp in candidates:
    size = int(sizes[comp])
    if size > MAX_RING_SIZE:
        rejected_big.append((int(comp), size))
        continue
    lo = int(np.searchsorted(sorted_labels, comp))
    hi = int(np.searchsorted(sorted_labels, comp, side="right"))
    idx = order[lo:hi]
    sub = graph[idx][:, idx]
    density = (sub.nnz / 2) / (size * (size - 1) / 2)
    if density < MIN_RING_DENSITY:
        rejected_sparse.append((int(comp), size, density))
        continue
    keep.append(comp)
```

Figure 4.3 — Ring detection: connected components filtered by size and density

4.2.4 ecdf() and rank aggregation

File: pipeline/09_score.py

Each feature becomes a percentile, each family the mean of its percentiles, and the families are ranked again before being combined into the consensus (mean) and any-family (maximum) scores.

pipeline/09_score.py

```python
def ecdf(values: np.ndarray, direction: str) -> np.ndarray:
    """Empirical CDF to [0, 1]. Ties share a percentile, as they should."""
    ranks = rankdata(values, method="average")
    scaled = (ranks - 0.5) / len(values)
    return scaled if direction == "high" else 1.0 - scaled


with common.stage("09_score: ECDF transform and rank aggregation"):
    column = {n: i for i, n in enumerate(names)}
    percentiles = {n: ecdf(matrix[:, column[n]], FEATURE_SPEC[n][1]) for n in kept}
    family_scores = {}
    for family in ("T", "B", "G"):
        members = [n for n in kept if FEATURE_SPEC[n][0] == family]
        family_scores[family] = np.mean([percentiles[n] for n in members], axis=0)
    # Aggregate ACROSS families by rank again, so a family with a heavy-tailed
    # internal distribution cannot dominate the other two.
    family_ranks = {f: (rankdata(v, method="average") - 0.5) / len(v)
                    for f, v in family_scores.items()}
    stacked = np.vstack(list(family_ranks.values()))
    # Two aggregations, because they answer different questions and the
    # measured family agreement is weak (Spearman 0.03-0.16 between families):
    #   score      consensus -- high only if SEVERAL independent families agree.
    #                           High precision, low recall, and the defensible one.
    #   score_any  max       -- high if ANY single family screams. Higher recall,
    #                           weaker evidence. Exposed so the dashboard can
    #                           switch between "strongest evidence" and "widest net".
    score = stacked.mean(axis=0)
    score_any = stacked.max(axis=0)
```

Figure 4.4 — ECDF transform and rank aggregation

4.3 Next.js dashboard

The dashboard (`web/`) is written in Next.js with shadcn/ui components and Recharts, and is built for a first-time reader: every page opens with a plain-language question and every chart has a caption saying what to look for. Its pages are:

- **Start here:** the headline numbers, the clearest example, and corpus-wide charts.
- **Groups:** all 1,381 rings, with a convergence plot and co-review graph for each.
- **Network:** the most suspicious 1% of accounts drawn as a network, in two views.
- **Copied reviews:** exact and near-duplicate clusters, shown side by side in full.
- **Accounts:** every account ranked by score, with a reviewing timeline and an explanation of which features drove its score.
- **Products:** products with the largest review bursts against the de-trended baseline.
- **How it works:** the method, the feature cull, the family agreement and the limits.

Chapter 5: Results

5.1 Results

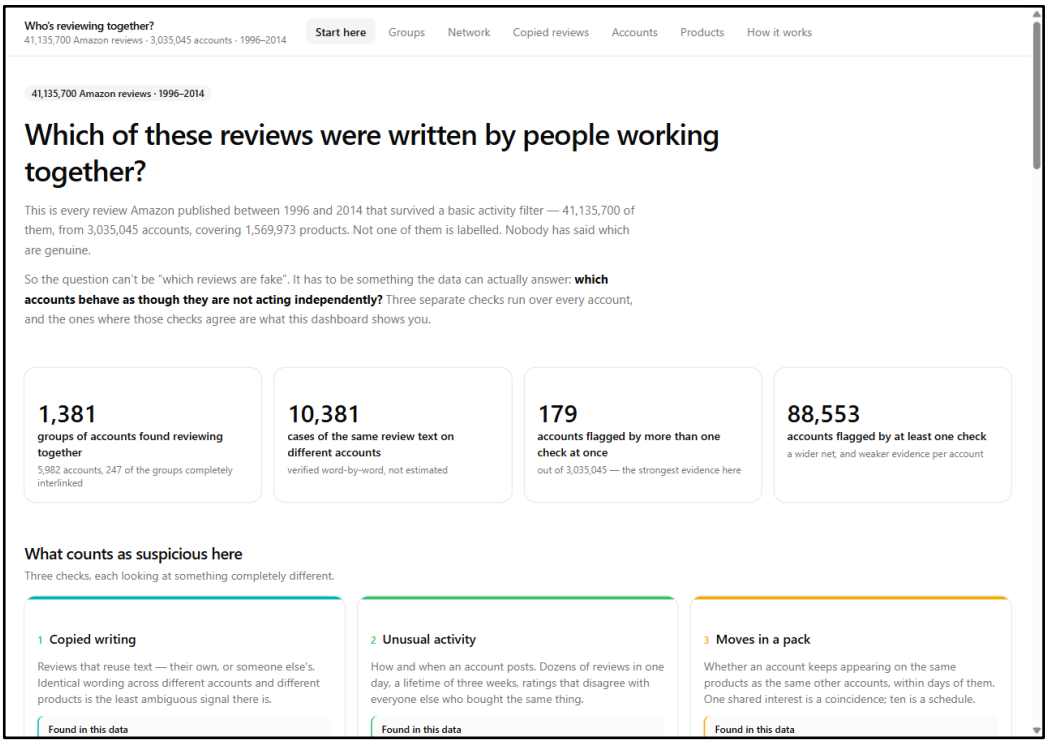


Figure 5.1 — Dashboard overview (Start here page)

Pipeline performance. The full build from the compressed source took about 48 minutes on the laptop, with a peak of 8.1 GB of RAM (Table 5.1). MinHash and the graph stage dominate; everything after ingest re-runs in minutes.

Stage	What it does	Time	Peak RAM
01 ingest	gzip NDJSON → Parquet, one pass	5.0 min	–
02 fact	surrogate keys, dimensions, fact table	26 s	6.2 GB
03 quality	data-quality report, daily baseline	12 s	1.3 GB
04 features	reviewer and product features	1.8 min	4.7 GB
05 duplicates	exact duplicate clusters	1.0 min	6.1 GB
06 minhash	MinHash signatures, LSH bands	18.5 min	66 MB / worker
07 neardup	candidates + exact Jaccard check	5.9 min	4.3 GB
08 graph	co-review graph, rings, centrality	14.4 min	8.1 GB
09 score	cull, ECDF, aggregation	48 s	2.3 GB
Total		**≈ 48 min**	**8.1 GB**

Table 5.1: Run time and peak memory of each pipeline stage

How suspicious is everybody? Figure 5.2 shows the consensus score of every account. The score is hard to max out, because it only rises when the families agree: 179 accounts (0.006%) score above 0.99. The any-family score is much wider, with 88,553 accounts above 0.99.

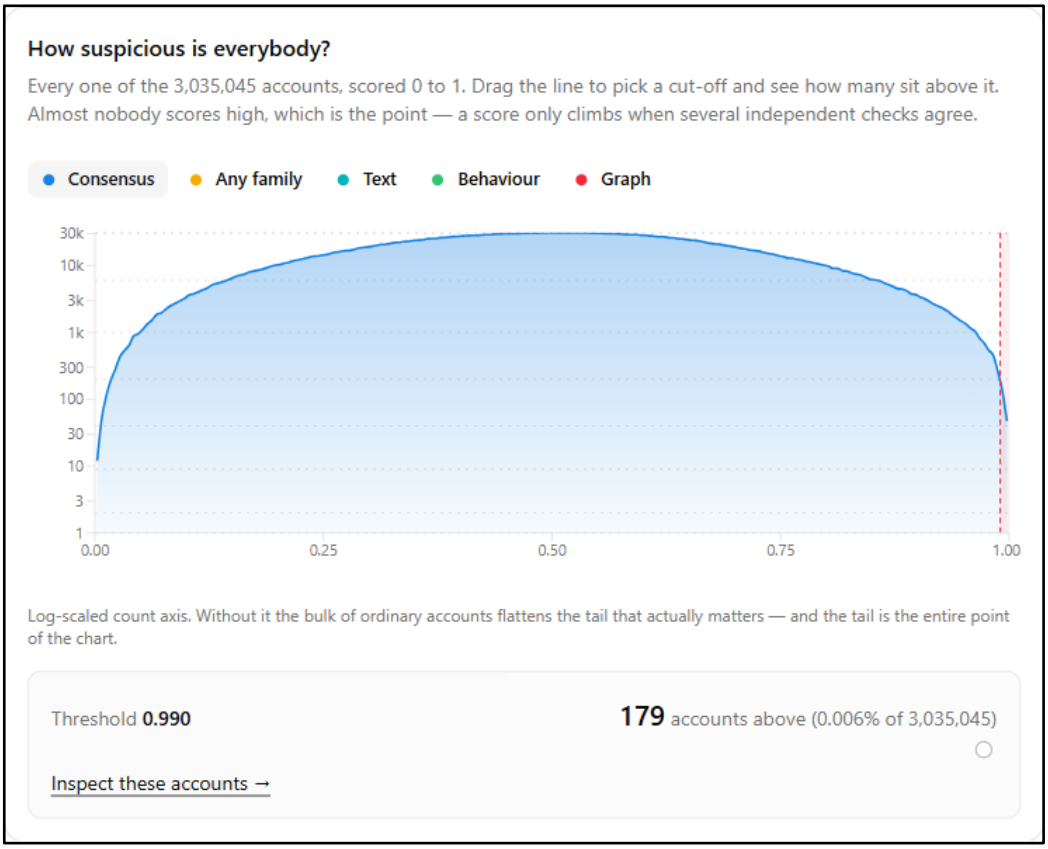


Figure 5.2 — Distribution of the suspicion score, with an adjustable cut-off

Why ratings are not used directly. 59.21% of all reviews are five-star (Table 3.1), so "gives lots of five-star reviews" describes most of Amazon. Ratings enter the score only as disagreement with other buyers of the same product.

Does the score mean anything? No rating level was ever shown to the scoring, yet the five-star share rises steadily from 48.6% in the least suspicious tenth of accounts to 66.0% in the most suspicious, while the one-star share falls from 8.8% to 4.3% (Figure 5.4). The score found this relationship on its own.

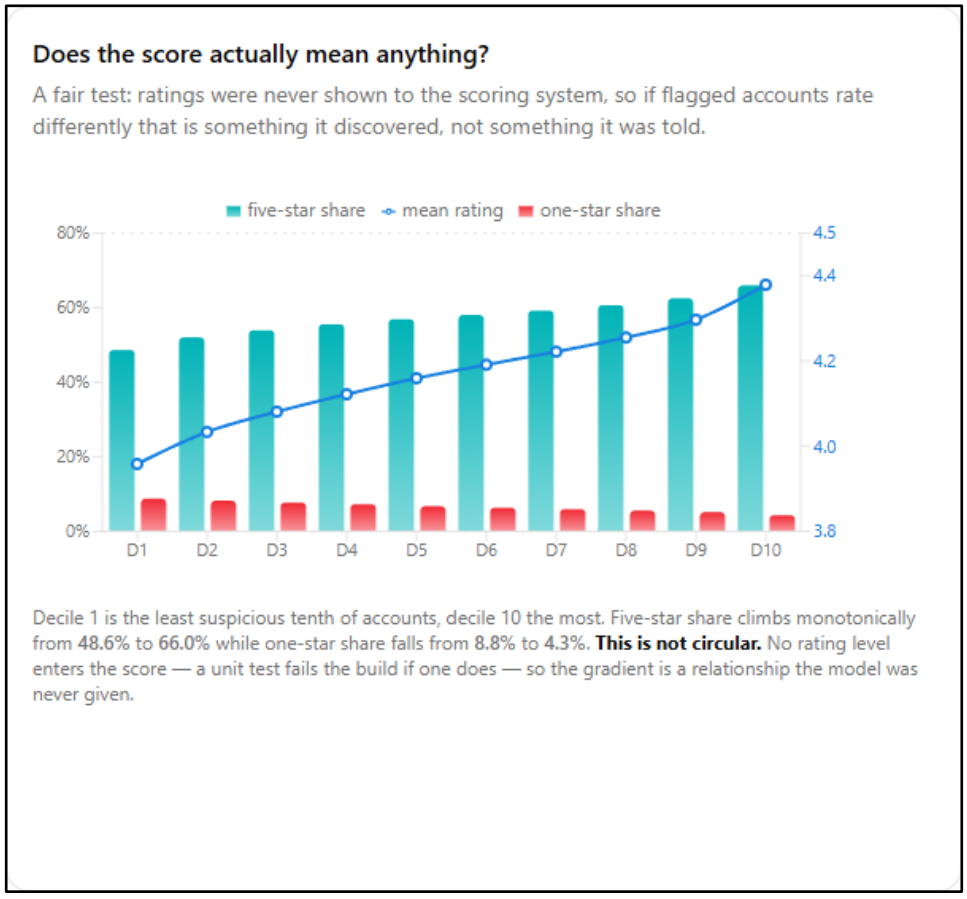


Figure 5.3 — Rating behaviour by suspicion decile

The three families mostly find different accounts. Taking each family's top 10,000 accounts, only 459 appear in more than one list and 10 in all three (Figure 5.4). The rank correlations between families are low (text–behaviour 0.162, text–graph 0.090, behaviour–graph 0.043), yet the overlap is well above chance: in the top 100,000 all three agree on 771 accounts against 108.6 expected by chance, a seven-fold lift.

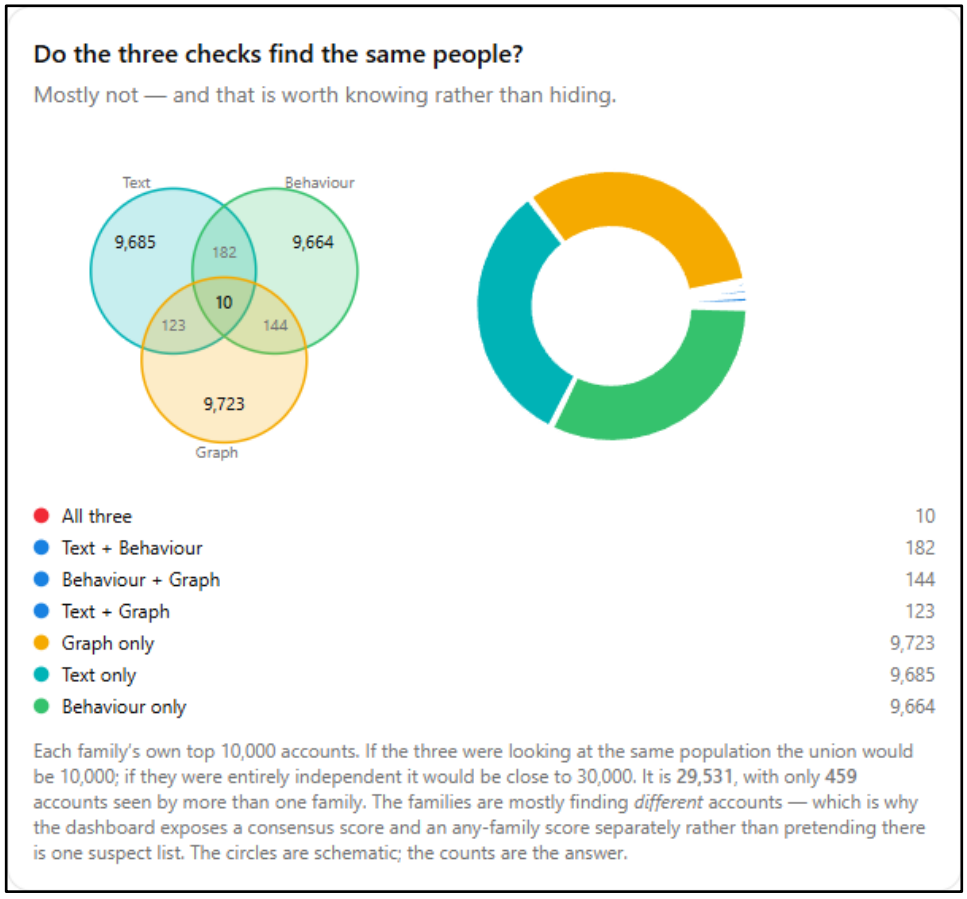


Figure 5.4 — Overlap between the three families' top 10,000 accounts

Groups reviewing together. The ring detector found **1,381 rings covering 5,982 accounts**, 247 of them completely interlinked (density 1.0). Figure 5.5 places every ring by lifespan and density.

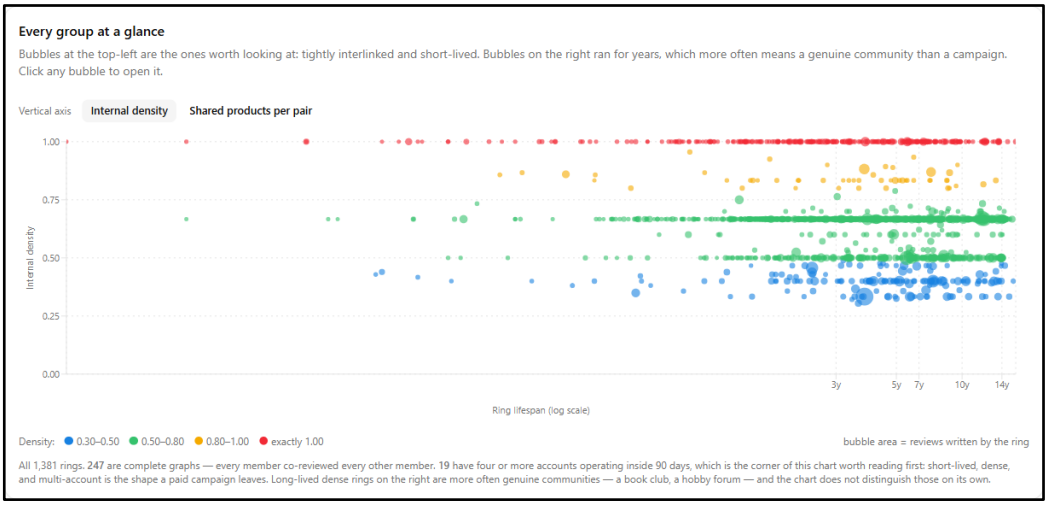


Figure 5.5 — All 1,381 rings by lifespan and density

The clearest case is ring #179919: six accounts, all six reviewing each of nineteen products, 79 reviews with a mean rating of 4.95, the whole operation inside 88 days. In the convergence plot (Figure 5.6) each row is a product and each dot a review; the vertical stacks are the ring. The

tightest product was hit by all six accounts within 23 days, all at five stars, and every member is linked to every other.

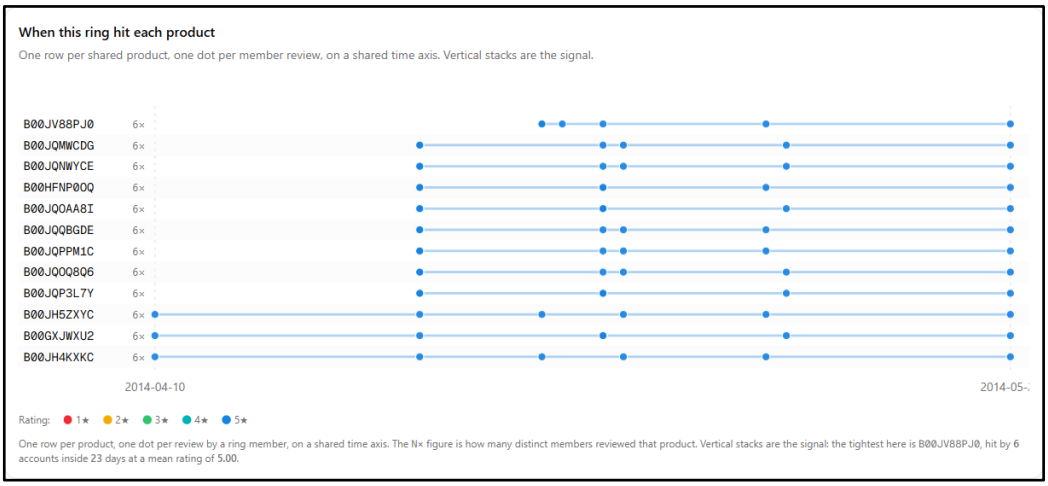


Figure 5.6 — Convergence plot of ring #179919

Copied text. Exact duplication is rare (Table 5.2): normalising the text more than doubles what is found, but still covers only 0.26% of reviews, and the largest exact clusters turned out to be Amazon's own review guidelines and filler text rather than campaigns. Near-duplicate detection is where the text signal lives: 79,938 verified clusters, 10,381 of them spanning different accounts.

Method	Clusters	Reviews	Notes
Exact, raw text	12,101	70,509	0.171% of reviews
Exact, normalised text	26,354	106,373	0.259% of reviews
Near-duplicate, loose ($J \geq 0.707$)	79,938	172,600	10,381 cross-account; 17.7% of candidates rejected
Near-duplicate, tight ($J \geq 0.878$)	41,874	87,236	7,898 cross-account; 20.6% rejected

Table 5.2: Exact and near-duplicate text detection

Cluster #60821 (Figure 5.7) is a typical templated campaign: five accounts posting the same long phone-case review on three products over 122 days, every one rated five stars, mean Jaccard similarity 0.965.

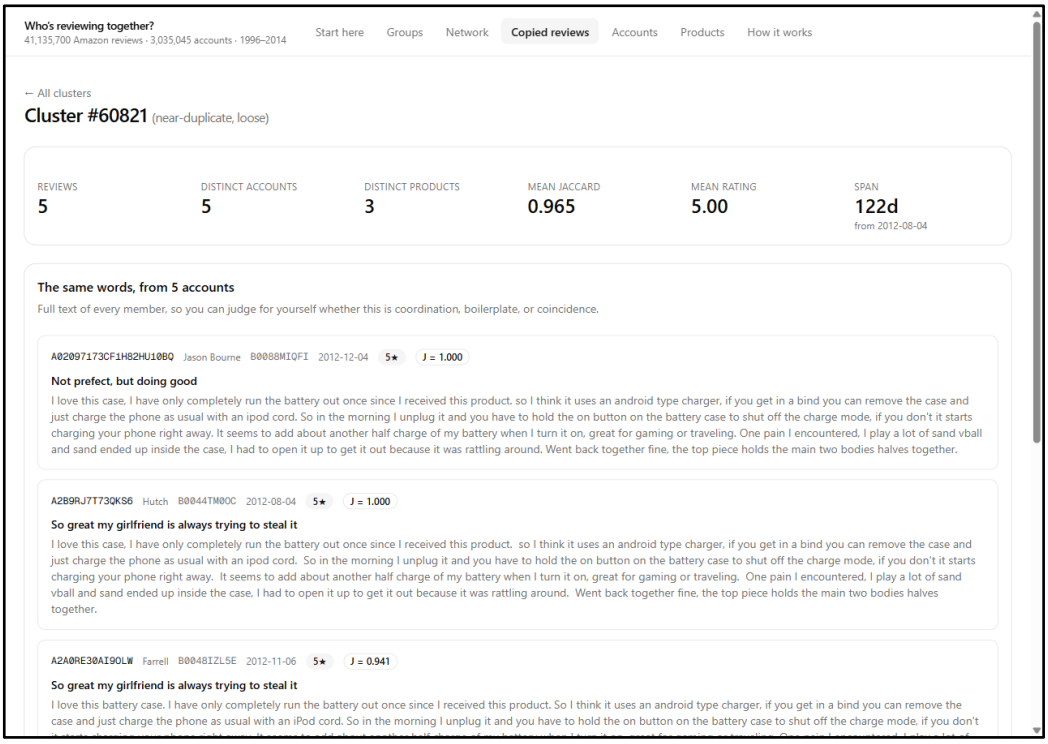


Figure 5.7 — Near-duplicate cluster #60821, shown side by side

The network of suspicious accounts. Restricted to the 30,378 accounts in the top 1% and linked by the ring rule, the network has 3,529 accounts and 17,730 links in 393 clusters. It splits into two shapes: 392 small separate groups holding 1,050 accounts (Figure 5.8), and one connected core of 2,479 accounts (Figure 5.9). Stricter linking rules were tried six ways, and the core always held 61–72% of the accounts.

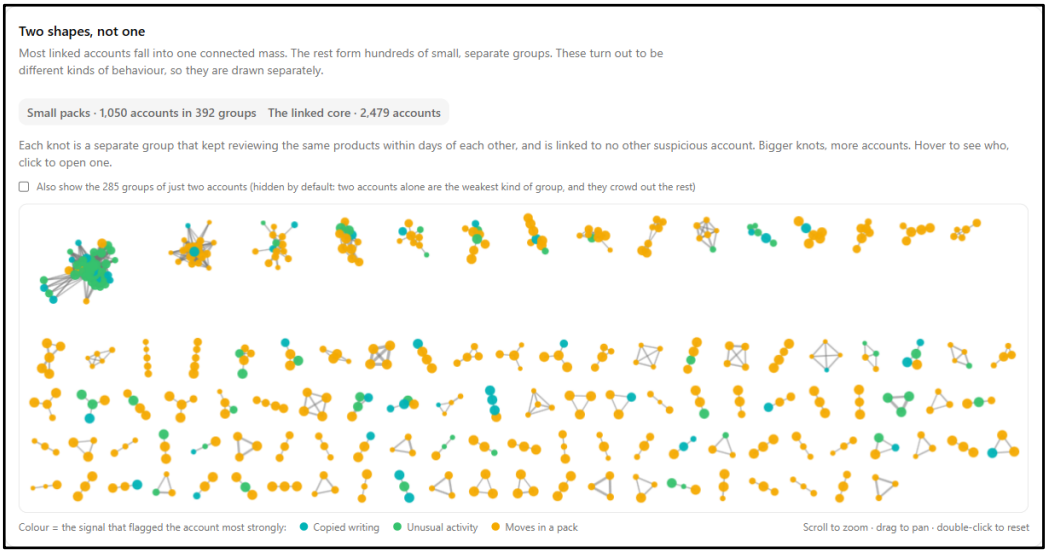


Figure 5.8 — Network view: the small, separate groups

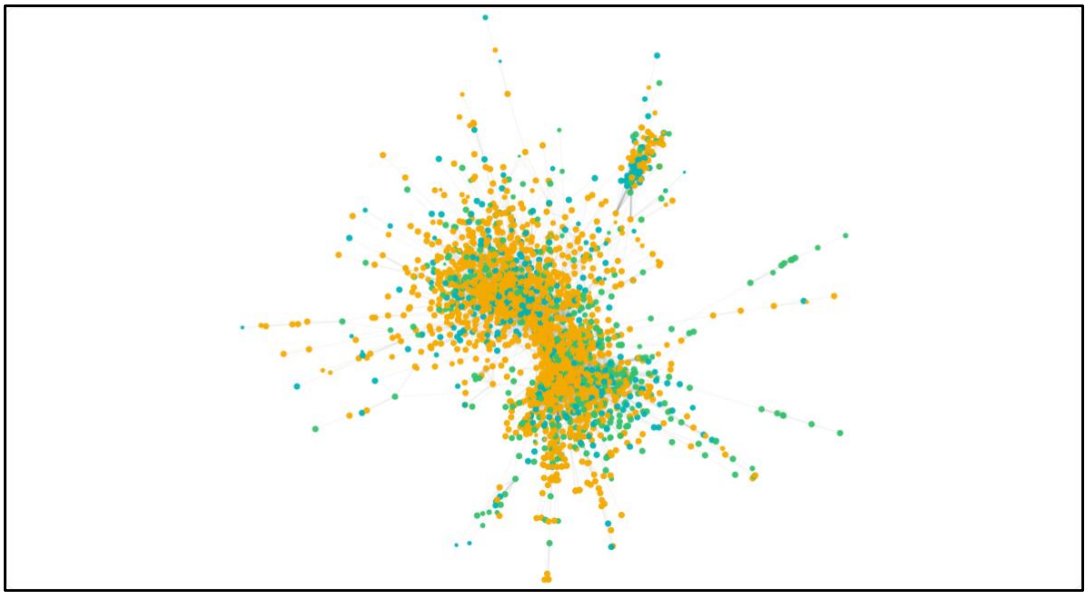


Figure 5.9 — Network view: the linked core of 2,479 accounts

Searching what the two groups wrote explains the difference (Table 5.3). Core accounts say they received the product free or discounted in exchange for the review 4.7 times as often as all 4,713 equally active accounts that are not suspicious; the small groups almost never do. The same score therefore flagged two different behaviours: reviewers who were all sent the same products, and groups coordinating quietly. Amazon's own Vine programme is not the explanation: it is mentioned in 0.17% of core reviews against 0.16% for the comparison group.

Group	Accounts	Reviews disclosing	Accounts disclosing
Linked core	2,479	1.68%	10.5%
Comparison (prolific, not suspicious)	4,713	0.36%	2.6%
Small groups	1,050	0.03%	0.2%

Table 5.3: Disclosure of free or discounted products, by group

Validation. The results of the three checks are summarised in Table 5.4. The cross-family check is strong: text features alone separate accounts that behaviour and graph rank as most and least suspicious with 72.1% accuracy. The helpful-vote proxy is weak. The planted-campaign study at first recovered almost nothing, and that is how it found a real bug: the co-review graph had a minimum of 20 reviews per account, which made the ring detector blind to every small account. Of the planted accounts it was allowed to see, it placed 92–100% into rings. Removing the floor grew the graph from 311,005 to 1,038,862 accounts and the rings from 314 to 1,381.

Check	Result	Reading
Cross-family (text predicts behaviour + graph)	Accuracy 0.721, ROC AUC 0.805	Families detect a common phenomenon
Helpful-vote proxy, consensus score	ROC AUC 0.562	Weak; the proxy is itself noisy
Helpful-vote proxy, text family	ROC AUC 0.658	Strongest single family
Helpful-vote proxy, graph family	ROC AUC 0.427	Below chance on this proxy
Planted campaigns (before fix)	Recall@10,000 = 0.0013	Exposed the 20-review graph floor

Table 5.4: Validation results

5.2 Discussion

The results show that coordinated reviewing can be surfaced at the scale of the full corpus on a single laptop, provided the work is arranged around the data: one pass over the compressed source, columnar storage, out-of-core SQL, and sparse matrices for the graph. The most convincing evidence is structural. Six accounts reviewing the same nineteen products within a month, every review five stars, is hard to explain as coincidence.

Two methodological corrections mattered more than any tuning. The first was recognising that a connected component is not a ring: the co-review graph never breaks apart, and calling its 44,039-account core a ring would have been the largest available error. The second came from validation: planted campaigns showed that an innocent-looking efficiency filter had made the detector blind to small accounts. Both were found by measuring, not by assuming.

The low agreement between families is itself a finding. Copying, unusual activity and pack behaviour mostly identify different accounts, which is why the dashboard offers both a high-precision consensus score and a wider any-family score rather than one suspect list. The network view adds a further distinction: the same score flags both incentivised reviewers and quietly coordinating groups.

5.3 Limitations

- No review in the corpus is labelled. The system identifies statistically unusual, coordinated behaviour; it does not establish that any account is fraudulent.
- The 5-core filter removed every account with fewer than five reviews, and the 2014 release has no verified-purchase flag or product metadata, so several published methods cannot be applied to this corpus at all.
- The helpful-vote validation uses a proxy, not ground truth, and its results are weak; the graph family scores below chance on it.
- The planted-campaign recall figures were measured before the graph fix they led to and have not yet been regenerated against the fixed detector.
- Bursts are de-trended against a single corpus-wide baseline rather than per product category, because no category data is available.

- Disclosure of free products explains only part of the linked core: only 10.5% of core accounts ever disclose, and undisclosed incentivised reviews cannot be seen. In 2014 disclosed incentivised reviews were still allowed by Amazon.
- Everything runs on one machine; the design was not tested on a multi-node cluster.

Chapter 6: Conclusion

This project built an end-to-end, fully local big-data pipeline that analyses all 41,135,700 reviews of the 2014 Amazon corpus for coordinated reviewing behaviour. The source is converted once into Parquet and processed out-of-core with DuckDB on a laptop with 15.4 GB of RAM, in about 48 minutes. Three independent families of evidence — copied text found with MinHash and LSH, behavioural anomalies against a de-trended baseline, and dense co-review rings — are combined by ECDF rank aggregation into an explained score for every one of 3,035,045 accounts. The system found 1,381 rings covering 5,982 accounts, 79,938 verified near-duplicate clusters and 179 accounts flagged by several families at once, and showed that the most suspicious accounts form two distinct shapes with different explanations. Validation without labels confirmed that the families detect a common phenomenon and uncovered a real blind spot, which was then fixed. A Next.js and shadcn/ui dashboard presents all of it for exploration, and a single command builds and runs the whole project.

Possible improvements include re-running the planted-campaign study against the fixed detector, adding per-category baselines if product metadata becomes available, testing streaming updates so new reviews are scored as they arrive, and measuring how the pipeline scales across a multi-node cluster.

Chapter 7: References

[1] J. McAuley, C. Targett, Q. Shi and A. van den Hengel, “Image-based recommendations on styles and substitutes,” in Proc. 38th Int. ACM SIGIR Conf. Research and Development in Information Retrieval, 2015, pp. 43–52.

[2] R. He and J. McAuley, “Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering,” in Proc. 25th Int. Conf. World Wide Web (WWW), 2016, pp. 507–517.

[3] N. Jindal and B. Liu, “Opinion spam and analysis,” in Proc. Int. Conf. Web Search and Data Mining (WSDM), 2008, pp. 219–230.

[4] G. Fei, A. Mukherjee, B. Liu, M. Hsu, M. Castellanos and R. Ghosh, “Exploiting burstiness in reviews for review spammer detection,” in Proc. 7th Int. AAAI Conf. Weblogs and Social Media (ICWSM), 2013.

[5] S. Xie, G. Wang, S. Lin and P. S. Yu, “Review spam detection via temporal pattern discovery,” in Proc. 18th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2012, pp. 823–831.

[6] E.-P. Lim, V.-A. Nguyen, N. Jindal, B. Liu and H. W. Lauw, “Detecting product review spammers using rating behaviors,” in Proc. 19th ACM Int. Conf. Information and Knowledge Management (CIKM), 2010, pp. 939–948.

[7] A. Mukherjee, B. Liu and N. Glance, “Spotting fake reviewer groups in consumer reviews,” in Proc. 21st Int. Conf. World Wide Web (WWW), 2012, pp. 191–200.

[8] A. Z. Broder, “On the resemblance and containment of documents,” in Proc. Compression and Complexity of Sequences, 1997, pp. 21–29.

[9] J. Leskovec, A. Rajaraman and J. D. Ullman, Mining of Massive Datasets, 3rd ed. Cambridge, U.K.: Cambridge University Press, 2020.

[10] L. Page, S. Brin, R. Motwani and T. Winograd, “The PageRank citation ranking: Bringing order to the web,” Stanford InfoLab, Tech. Rep., 1999.

[11] M. Raasveldt and H. Mühleisen, “DuckDB: An embeddable analytical database,” in Proc. ACM SIGMOD Int. Conf. Management of Data, 2019, pp. 1981–1984.

[12] S. Kumar, B. Hooi, D. Makhija, M. Kumar, C. Faloutsos and V. S. Subrahmanian, “REV2: Fraudulent user prediction in rating platforms,” in Proc. 11th ACM Int. Conf. Web Search and Data Mining (WSDM), 2018, pp. 333–341.

[13] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng and P. S. Yu, “Enhancing graph neural network-based fraud detectors against camouflaged fraudsters,” in Proc. 29th ACM Int. Conf. Information and Knowledge Management (CIKM), 2020, pp. 315–324.

[14] S. Rayana and L. Akoglu, “Collective opinion spam detection: Bridging review networks and metadata,” in Proc. 21st ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2015, pp. 985–994.

[15] Apache Software Foundation, “Apache Parquet documentation.” [Online]. Available: https://parquet.apache.org/docs/

[16] Vercel, “Next.js documentation.” [Online]. Available: https://nextjs.org/docs